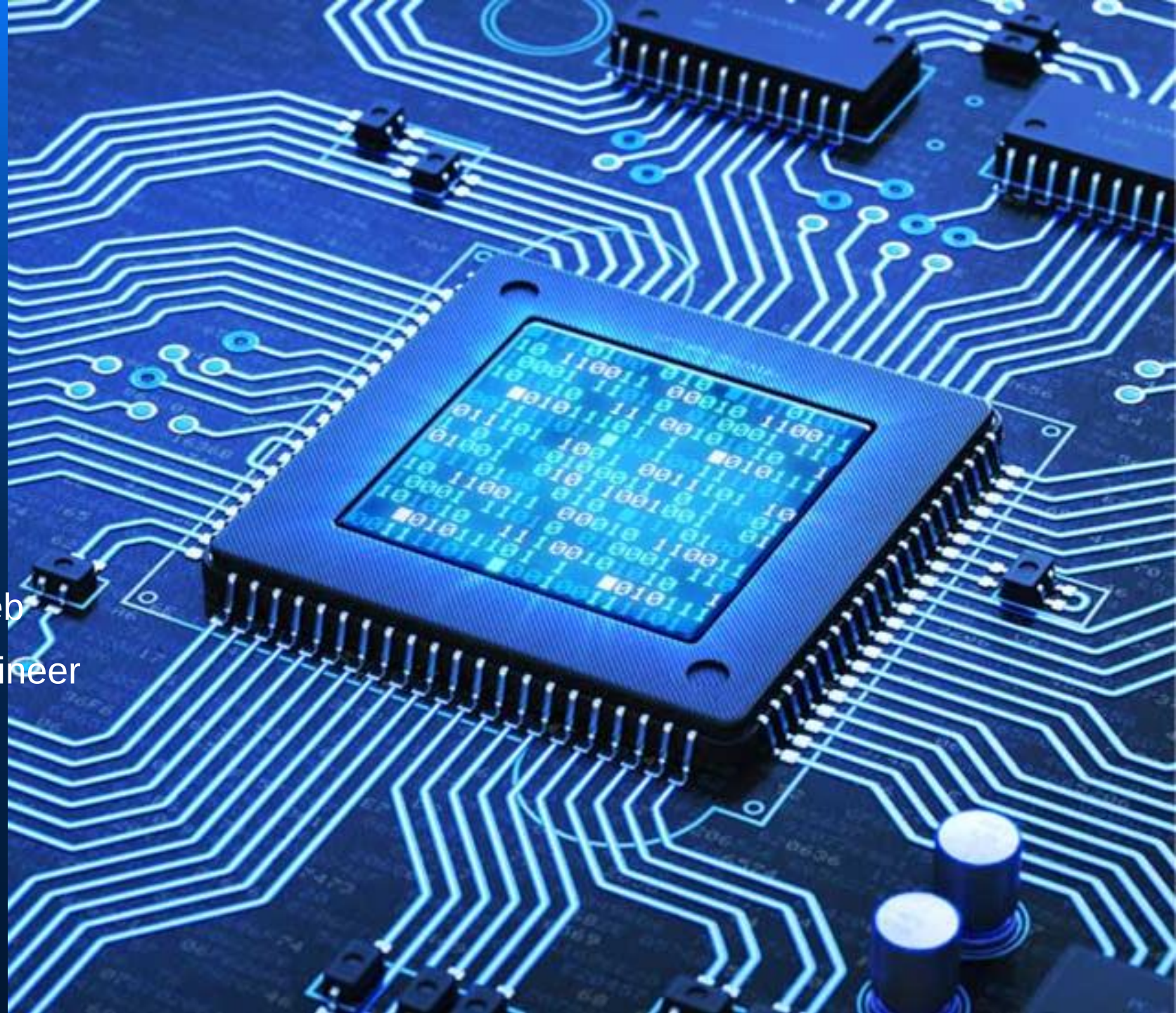
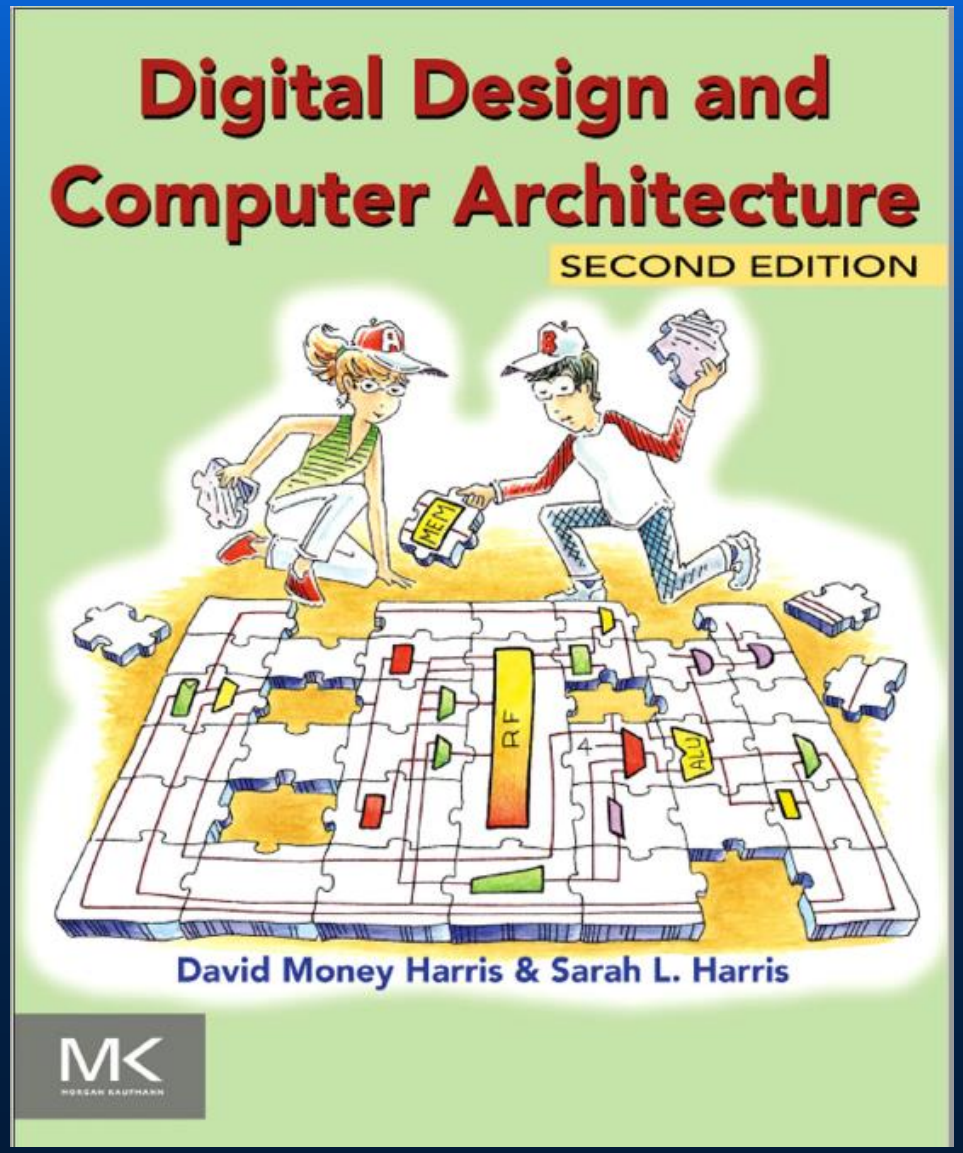
# Computer Architecture Session 2

By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer






**This presentation is prepared from:**

**Digital Design and Computer Architecture, 2nd Edition.**
**David Harris and Sarah L. Harris.**

# Agenda

## Chapter 6: Architecture

- # 4. Programming
- # 6.1 Memory Map
- # 6.2 Program Start

# 4. Programming

# 4. Programming

- Many high-level languages use common software constructs such as **arithmetic** and **logical operations**, if/else statements, for and while loops, array indexing, and function calls.
- We explore how to translate these high-level constructs into MIPS assembly code.
    - Arithmetic/Logical Instructions
    - Branching
    - Conditional Statements
    - Getting Loopy
    - Arrays
    - Function Calls

# 4. Programming


# 4.1 Arith/Logical Instructions

# 4.2 Branching

# 4.3 Conditional Statements

# 4.4 Getting Loopy

# 4.5 Arrays

# 4.6 Function Calls

# 4.1 Arithmetic/Logical Instructions

- The MIPS architecture defines a variety of arithmetic and logical instructions.

**Logical Instructions**

- MIPS logical operations include and, or, xor, and nor.
- These R-type instructions operate bit-by-bit on two source registers and write the result to the destination register.
- Logical operations can also operate on **immediates**.
- These I-type instructions are andi, ori, and xori.
- nori is not supported.

# 4.1 Arithmetic/Logical Instructions

Example:

- Use (and with 0) to mask bits (force unwanted bits to 0).
- Use (or with 0) instruction to combine bits from two registers.
- MIPS doesn't support a NOT instruction, so use (NOR with 0) instead.

Assembly Code

```
and $s3, $s1, $s2
or  $s4, $s1, $s2
xor $s5, $s1, $s2
nor $s6, $s1, $s2
```

Source Registers

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| $s1 | 1111 | 1111 | 1111 | 1111 | 0000 | 0000 | 0000 | 0000 |
| $s2 | 0100 | 0110 | 1010 | 0001 | 1111 | 0000 | 1011 | 0111 |

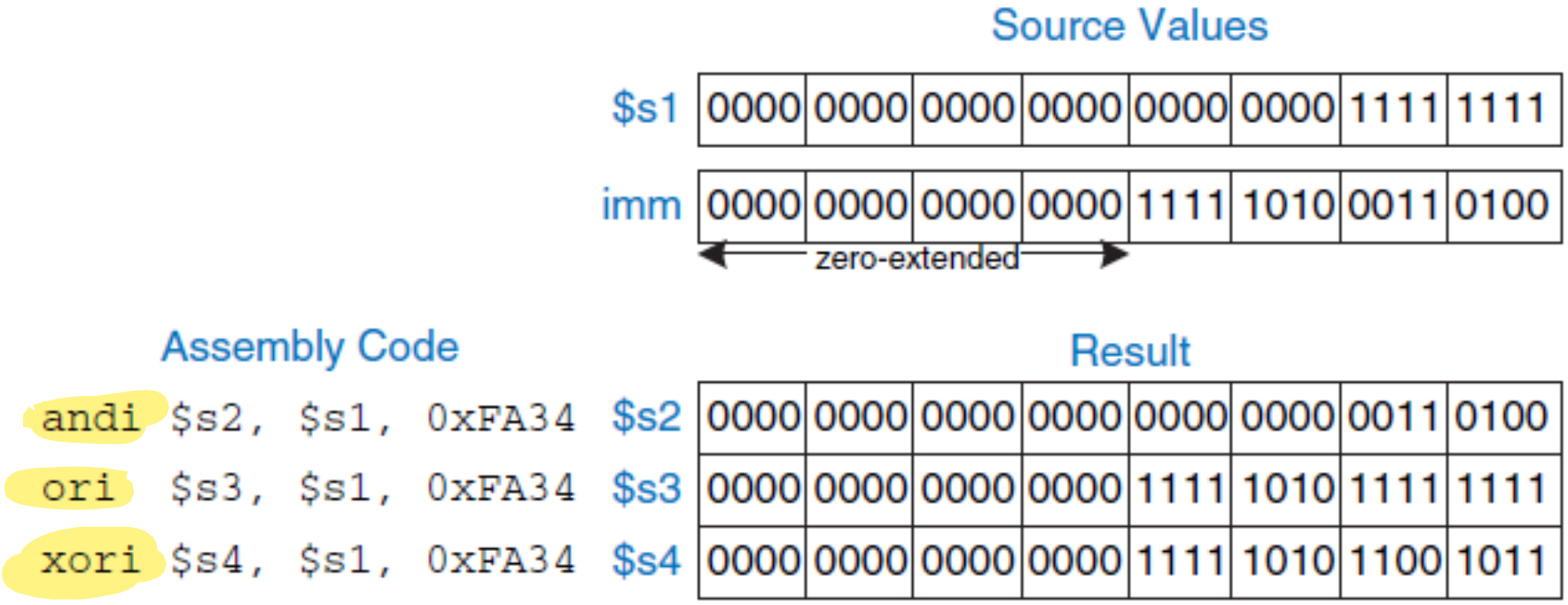
Result

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| $s3 | 0100 | 0110 | 1010 | 0001 | 0000 | 0000 | 0000 | 0000 |
| $s4 | 1111 | 1111 | 1111 | 1111 | 1111 | 0000 | 1011 | 0111 |
| $s5 | 1011 | 1001 | 0101 | 1110 | 1111 | 0000 | 1011 | 0111 |
| $s6 | 0000 | 0000 | 0000 | 0000 | 0000 | 1111 | 0100 | 1000 |

# 4.1 Arithmetic/Logical Instructions

Example:

- Remember: The immediate is zero-extended to 32 bits.



# 4.1 Arithmetic/Logical Instructions

Shift Instructions:

- Shift instructions shift the value in a register left or right by up to 31 bits.
- Shift operations multiply or divide by powers of two.
- MIPS shift operations are **sll** (shift left logical), **srl** (shift right logical), and **sra** (shift right arithmetic).
- Left shifts always fill the least significant bits with 0's.However, right shifts can be either logical (0's shift into the MSB) or arithmetic (the sign bit shifts into the MSB).

# 4.1 Arithmetic/Logical Instructions

## Shift Instructions:

- Example: the machine code for the R-type instructions sll, srl, and sra.

| Assembly Code | Field Values | | | | | | Machine Code | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | op | rs | rt | rd | shamt | funct | op | rs | rt | rd | shamt | funct | |
| sll $t0, $s1, 4 | 0 | 0 | 17 | 8 | 4 | 0 | 000000 | 00000 | 10001 | 01000 | 00100 | 000000 | (0x00114100) |
| srl $s2, $s1, 4 | 0 | 0 | 17 | 18 | 4 | 2 | 000000 | 00000 | 10001 | 10010 | 00100 | 000010 | (0x00119102) |
| sra $s3, $s1, 4 | 0 | 0 | 17 | 19 | 4 | 3 | 000000 | 00000 | 10001 | 10011 | 00100 | 000011 | (0x00119903) |
| | 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits | 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits | |

- Shifting left by N >> multiplying it by $2^N$.
- Arithm shifting right by N >> dividing it by $2^N$.

# 4.1 Arithmetic/Logical Instructions

## Shift Instructions:

- MIPS also has variable-shift instructions: **sllv** (shift left logical variable), **srlv** (shift right logical variable), and **srav** (shift right arithmetic variable).
- Example:

| Assembly Code | op | rs | rt | rd | shamt | funct | op | rs | rt | rd | shamt | funct | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Field Values | | | | | | Machine Code | | | | | | |
| sllv $s3, $s1, $s2 | 0 | 18 | 17 | 19 | 0 | 4 | 000000 | 10010 | 10001 | 10011 | 00000 | 000100 | (0x02519804) |
| srlv $s4, $s1, $s2 | 0 | 18 | 17 | 20 | 0 | 6 | 000000 | 10010 | 10001 | 10100 | 00000 | 000110 | (0x0251A006) |
| srav $s5, $s1, $s2 | 0 | 18 | 17 | 21 | 0 | 7 | 000000 | 10010 | 10001 | 10101 | 00000 | 000111 | (0x0251A807) |
| | 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits | 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits | |

Source Values

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| $s1 | 1111 | 0011 | 0000 | 0100 | 0000 | 0010 | 1010 | 1000 |
| $s2 | 0000 | 0000 | 0000 | 0000 | 0000 | 0000 | 0000 | **1000** |

| Assembly Code | Result | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| sllv $s3, $s1, $s2 | $s3 | 0000 | 0100 | 0000 | 0010 | 1010 | 1000 | 0000 | 0000 |
| srlv $s4, $s1, $s2 | $s4 | 0000 | 0000 | 1111 | 0011 | 0000 | 0100 | 0000 | 0010 |
| srav $s5, $s1, $s2 | $s5 | 1111 | 1111 | 1111 | 0011 | 0000 | 0100 | 0000 | 0010 |

# 4.1 Arithmetic/Logical Instructions

Generating Constants Instructions:

- Use **addi** instruction to assign 16-bit constants.

**Example:** 16-BIT CONSTANT

| High-Level Code | MIPS Assembly Code |
|---|---|
| int a = 0x4f3c; | # $s0 = a<br>addi $s0, $0, 0x4f3c # a = 0x4f3c |

- To assign 32-bit constants, use load upper immediate (**lui**) + or immediate (**ori**).
- **lui** loads a 16-bit immediate into the upper half of a register and sets the lower half to 0.
- **ori** merges a 16-bit immediate into the lower half.

**Example:** 32-BIT CONSTANT

| High-Level Code | MIPS Assembly Code |
|---|---|
| int a = 0x6d5e4f3c; | # $s0 = a<br>lui $s0, 0x6d5e # a = 0x6d5e0000<br>ori $s0, $s0, 0x4f3c # a = 0x6d5e4f3c |

# 4.1 Arithmetic/Logical Instructions

Multiplication and Division Instructions:

- Multiplying two 32-bit numbers produces a 64-bit product.
- Dividing two 32-bit numbers produces a 32-bit quotient and a 32-bit remainder.
- The MIPS architecture has two special-purpose registers, hi and lo, which are used to hold the results of multiplication and division.
- mult $s0, $s1 multiplies the values in $s0 and $s1. The 32 most significant bits of the product are placed in hi and the 32 least significant bits are placed in lo.
- Similarly, div $s0, $s1 computes $s0/$s1. The quotient is placed in lo and the remainder is placed in hi.

# 4.1 Arithmetic/Logical Instructions

## Multiplication and Division Instructions:

- hi and lo are not among the usual 32 MIPS registers, so special instructions are needed to access them.
- mfhi $s2 (move from hi) copies the value in hi to $s2.
- mflo $s3 (move from lo) copies the value in lo to $s3.
- MIPS provides another multiply instruction that produces a 32-bit result in a general purpose register. mul $s1, $s2, $s3 multiplies the values in $s2 and $s3 and places the 32-bit result in $s1.

# 4. Programming


# 4.1 Arith/Logical Instructions

# 4.2 Branching

# 4.3 Conditional Statements

# 4.4 Getting Loopy

# 4.5 Arrays

# 4.6 Function Calls

# 4.2 Branching

- An advantage of a computer is its ability to make decisions.
- A computer performs different tasks depending on the input.
  For example, if/else statements, switch/case statements, while loops, and for loops all conditionally execute code depending on some test.
- To sequentially execute instructions, the program counter inc by 4 after each instruction.
- Branch instructions modify PC to *skip over sections of code* or to *repeat previous code*.
- Conditional branch instructions perform a test and branch only if the test is TRUE.
- Unconditional branch instructions, called jumps, always branch.

# 4.2 Branching

Conditional Branches:

- MIPS has two conditional branches: branch if equal (**beq**) and branch if not equal (**bne**).

Example:

- Note that branches are written as beq rs, rt, imm, where rs is the first source register. This order is reversed from most I-type instructions.
- Assembly code uses labels for instruction locations which are translated into instruction addresses in the machine code.

**MIPS Assembly Code**

```
    addi $s0, $0, 4          # $s0 = 0 + 4 = 4
    addi $s1, $0, 1          # $s1 = 0 + 1 = 1
    sll $s1, $s1, 2          # $s1 = 1 << 2 = 4
    beq $s0, $s1, target     # $s0 == $s1, branch is taken
    addi $s1, $s1, 1             # not executed
    sub $s1, $s1, $s0            # not executed
target:
    add $s1, $s1, $s0 # $s1 = 4 + 4 = 8
```

# 4.2 Branching

Jump:

- MIPS has 3 unconditional branches: jump (**j**), jump and link (**jal**), and jump register (**jr**).
- Jump (**j**) jumps directly to the instruction at the specified label.
- Jump and link (**jal**) is like **j** but is used by functions to save a return address.
- Jump register (**jr**) jumps to the address held in a register.

Example: The jump instruction (j).

**MIPS Assembly Code**

```
    addi $s0, $0, 4          # $s0 = 4
    addi $s1, $0, 1          # $s1 = 1
    j target                 # jump to target
    addi $s1, $s1, 1         # not executed
    sub $s1, $s1, $s0        # not executed
target:
    add $s1, $s1, $s0        # $s1 = 1 + 4 = 5
```

# 4.2 Branching

Jump:

Example: The jump register instruction (**jr**).

- Instruction addresses are given to the left of each instruction.
- jr $s0 jumps to the address held in $s0, 0x00002010.

| MIPS Assembly Code | |
|---|---|
| 0x00002000 addi $s0, $0, 0x2010 | # $s0 = 0x2010 |
| 0x00002004 jr $s0 | # jump to 0x00002010 |
| 0x00002008 addi $s1, $0, 1 | # not executed |
| 0x0000200c sra $s1, $s1, 2 | # not executed |
| 0x00002010 lw $s3, 44($s1) | # executed after jr instruction |

# 4. Programming


# 4.1 Arith/Logical Instructions

# 4.2 Branching

# 4.3 Conditional Statements

# 4.4 Getting Loopy

# 4.5 Arrays

# 4.6 Function Calls

# 4.3 Conditional Statements

## If Statements:

| High-Level Code | MIPS Assembly Code |
|---|---|
| if (i == j)<br>  f = g + h;<br>f = f – i; | # $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j<br>  **bne** $s3, $s4, L1  # if i != j, skip if block<br>  add $s0, $s1, $s2  # if block: f = g + h<br>L1:<br>  sub $s0, $s0, $s3  # f = f − i |

## If/Else Statements:

| High-Level Code | MIPS Assembly Code |
|---|---|
| if (i == j)<br>  f = g + h;<br>else<br>  f = f – i; | # $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j<br>  **bne** $s3, $s4, else  # if i != j, branch to else<br>  add $s0, $s1, $s2  # if block: f = g + h<br>  j L2  # skip past the else block<br>else:<br>  sub $s0, $s0, $s3  # f = f − i<br>L2: |

# 4.3 Conditional Statements

## Switch/Case Statements:

| High-Level Code | MIPS Assembly Code |
|---|---|
| switch (amount) {<br>  case 20: fee = 2; break;<br>  case 50: fee = 3; break;<br>  default: fee = 0;<br>}<br>// equivalent func using if/else statements<br>if (amount == 20) fee = 2;<br>else if (amount == 50) fee = 3;<br>else fee = 0; | # $s0 = amount, $s1 = fee<br>case20:<br>  addi $t0, $0, 20 # $t0 = 20<br>  bne $s0, $t0, case50 # amount != 20? skip to case50<br>  addi $s1, $0, 2 # if so, fee = 2<br>  j done # break out of case<br>case50:<br>  addi $t0, $0, 50 # $t0 = 50<br>  bne $s0, $t0, default # amount != 50? skip to default<br>  addi $s1, $0, 3 # if so, fee = 3<br>  j done # break out of case<br>default:<br>  add $s1, $0, $0 # fee = 0<br>done: |

# 4. Programming


# 4.1 Arith/Logical Instructions

# 4.2 Branching

# 4.3 Conditional Statements

# 4.4 Getting Loopy

# 4.5 Arrays

# 4.6 Function Calls

# 4.4 Getting Loopy

While Loops:

Example: Calculate the value of x such that $2^x = 128$.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int pow = 1;<br>int x = 0;<br>while (pow != 128)<br>{<br>  pow = pow * 2;<br>  x = x + 1;<br>} | # $s0 = pow, $s1 = x<br>addi $s0, $0, 1  # pow = 1<br>addi $s1, $0, 0  # x = 0<br>addi $t0, $0, 128  # t0 = 128 for comparison<br>while:<br>  beq $s0, $t0, done  # if pow == 128, exit while loop<br>  sll $s0, $s0, 1  # pow = pow * 2<br>  addi $s1, $s1, 1  # x = x + 1<br>  j while<br>done: |

# 4.4 Getting Loopy

For Loops:

Example: Add the numbers from 0 to 9.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int sum = 0;<br>for (i = 0; i != 10; i = i + 1) {<br>  sum = sum + i ;<br>}<br>// equivalent to the following while loop<br>int sum = 0;<br>int i = 0;<br>while (i != 10) {<br>  sum = sum + i;<br>  i = i + 1;<br>} | # $s0 = i, $s1 = sum<br>add $s1, $0, $0  # sum = 0<br>addi $s0, $0, 0  # i = 0<br>addi $t0, $0, 10  # $t0 = 10<br>for:<br>  beq $s0, $t0, done  # if i == 10, branch to done<br>  add $s1, $s1, $s0  # sum = sum + i<br>  addi $s0, $s0, 1  # increment i<br>  j for<br>done: |

# 4.4 Getting Loopy

## Magnitude Comparison:

- MIPS provides the *set less than* instruction, **slt**, for magnitude comparison.
- **slt** sets rd to 1 when rs < rt. Otherwise, rd is 0.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int sum = 0;<br>for (i = 1; i < 101; i = i * 2)<br>  sum = sum + i; | # $s0 = i, $s1 = sum<br>addi $s1, $0, 0  # sum = 0<br>addi $s0, $0, 1  # i = 1<br>addi $t0, $0, 101  # $t0 = 101<br>loop:<br>  **slt** $t1, $s0, $t0  # if (i < 101) $t1 = 1, else $t1 = 0<br>  **beq** $t1, $0, done  # if $t1 == 0 (i >= 101), branch to done<br>  add $s1, $s1, $s0  # sum = sum + i<br>  sll $s0, $s0, 1  # i = i * 2<br>  **j** loop<br>done: |

# 4. Programming


# 4.1 Arith/Logical Instructions

# 4.2 Branching

# 4.3 Conditional Statements

# 4.4 Getting Loopy

# 4.5 Arrays

# 4.6 Function Calls

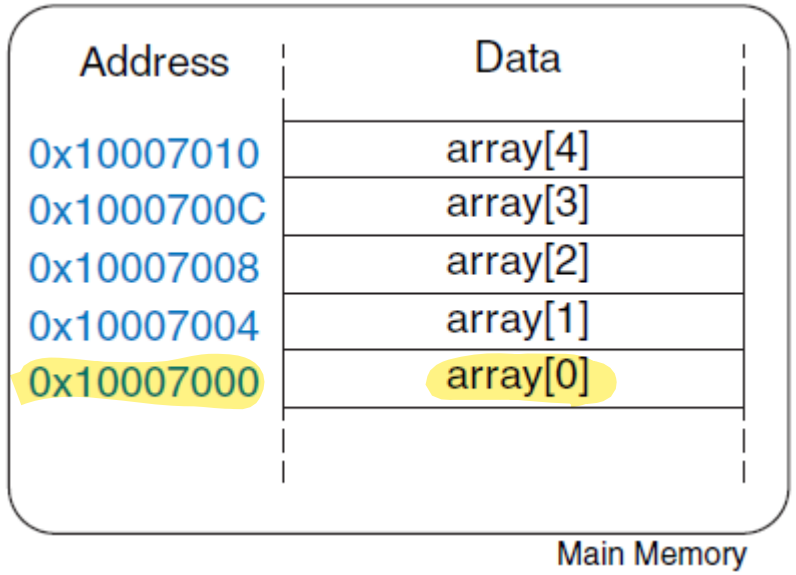
# 4.5 Arrays

## Arrays:

- Arrays are useful for accessing large amounts of similar data.
- An array is organized as sequential data addresses in memory.

## Array Indexing

- The index ranges from 0 to 4. In this case, the array is stored in a processor's main memory starting at base address 0x10007000.



# 4.5 Arrays

Example: Multiplie the first two elements in array by 8 and stores them back into the array.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int array[5];<br><br>array[0] = array[0] * 8;<br><br>array[1] = array[1] * 8; | # $s0 = base address of array<br>lui $s0, 0x1000 # $s0 = 0x10000000<br>ori $s0, $s0, 0x7000 # $s0 = 0x10007000<br><br>lw $t1, 0($s0) # $t1 = array[0]<br>sll $t1, $t1, 3 # $t1 = $t1 << 3 = $t1 * 8<br>sw $t1, 0($s0) # array[0] = $t1<br><br>lw $t1, 4($s0) # $t1 = array[1]<br>sll $t1, $t1, 3 # $t1 = $t1 << 3 = $t1 * 8<br>sw $t1, 4($s0) # array[1] = $t1 |

# 4.5 Arrays

Example: Use a for loop to multiply by 8 a 1000-element array stored in the memory.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int i;<br><br>int array[1000];<br><br>for (i = 0; i < 1000; i = i + 1)<br>  array[i] = array[i] * 8; | # $s0 = array base address, $s1 = i<br>lui $s0, 0x23B8  # $s0 = 0x23B80000<br>ori $s0, $s0, 0xF000  # $s0 = 0x23B8F000<br>addi $s1, $0, 0  # i = 0<br>addi $t2, $0, 1000  # $t2 = 1000<br>loop:<br>  slt $t0, $s1, $t2  # i < 1000?<br>  beq $t0, $0, done  # if not, then done<br>  sll $t0, $s1, 2  # $t0 = i*4 (byte offset)<br>  add $t0, $t0, $s0  # address of array[i]<br>  lw $t1, 0($t0)  # $t1 = array[i]<br>  sll $t1, $t1, 3  # $t1 = array[i] * 8<br>  sw $t1, 0($t0)  # array[i] = array[i] * 8<br>  addi $s1, $s1, 1  # i = i + 1<br>  j loop  # repeat<br>done: |

# 4.5 Arrays

## Bytes and Characters:

- MIPS provides load byte and store byte instructions to manipulate bytes or characters of data: load byte unsigned (**lbu**), load byte (**lb**), and store byte (**sb**).

Example: converts a ten-entry array of characters from lowercase to upper-case.

| High-Level Code | MIPS Assembly Code |
|---|---|
| // high-level code<br>char chararray[10];<br>int i;<br>for (i = 0; i != 10; i = i + 1)<br>  chararray[i] = chararray[i] – 32; | # $s0 = base address of chararray, $s1 = i<br>  addi $s1, $0, 0  # i = 0<br>  addi $t0, $0, 10  # $t0 = 10<br>loop: beq $t0, $s1, done  # if i == 10, exit loop<br>  add $t1, $s1, $s0  # $t1 = address of chararray[i]<br>  lb $t2, 0($t1)  # $t2 = array[i]<br>  addi $t2, $t2, –32  # convert to upper case<br>  sb $t2, 0($t1)  # store new value in array<br>  addi $s1, $s1, 1  # i = i+1<br>  j loop  # repeat<br>done: |

# 4. Programming


# 4.1 Arith/Logical Instructions

# 4.2 Branching

# 4.3 Conditional Statements

# 4.4 Getting Loopy

# 4.5 Arrays

# 4.6 Function Calls

# 4.6 Function Calls

- High-level languages often use functions (procedures) to reuse frequently accessed code.
- Functions have inputs, called arguments, and an output, called the return value.
- In MIPS, the **caller** places up to 4 arguments in registers $a0–$a3 before making the function call, and the **callee** places the return value in registers $v0–$v1 before finishing.
- The caller stores the return address in $ra and jumps to the callee using the jump and link instruction (**jal**).
- The callee must not overwrite any architectural state or memory that the caller is depending on (the saved registers, $s0–$s7, $ra, and the stack, a portion of memory used for temporary variables).

# 4.6 Function Calls

- The **caller** places up to 4 arguments in registers $a0–$a3 before making the function call, and the **callee** places the return value in registers $v0–$v1 before finishing.
- The caller stores the return address in $ra.

| Name | Number | Use |
|---|---|---|
| $0 | 0 | the constant value 0 |
| $at | 1 | assembler temporary |
| **$v0–$v1** | **2–3** | **function return value** |
| **$a0–$a3** | **4–7** | **function arguments** |
| $t0–$t7 | 8–15 | temporary variables |
| $s0–$s7 | 16–23 | saved variables |
| $t8–$t9 | 24–25 | temporary variables |
| $k0–$k1 | 26–27 | operating system (OS) temporaries |
| $gp | 28 | global pointer |
| $sp | 29 | stack pointer |
| $fp | 30 | frame pointer |
| **$ra** | **31** | **function return address** |

# 4.6 Function Calls

## Function Calls and Returns

- MIPS uses the jump and link instruction (**jal**) to call a function and the jump register instruction (**jr**) to return from a function.

Example: Simple function is called with no input arguments and generates no return value.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int main() {<br>  simple();<br>  . . .<br>}<br>// void means the function returns no value<br>void simple() {<br>  return;<br>} | 0x00400200 main: **jal** simple  # call function<br>0x00400204 . . .<br><br>0x00401020 simple: **jr** $ra  # return |

# 4.6 Function Calls

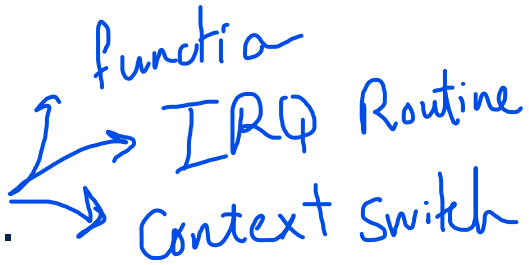
## Input Arguments and Return Values

Example: Function *diffofsums* is called with four arguments and returns one result.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int main()<br>{<br>  int y;<br>  . . .<br>  y = diffofsums(2, 3, 4, 5);<br>  . . .<br>}<br>int diffofsums(int f, int g, int h, int i)<br>{<br>  int result;<br>  result = (f + g) − (h + i);<br>  Return<br>} | main:<br>  addi $a0, $0, 2  # argument 0 = 2<br>  addi $a1, $0, 3  # argument 1 = 3<br>  addi $a2, $0, 4  # argument 2 = 4<br>  addi $a3, $0, 5  # argument 3 = 5<br>  **jal** diffofsums  # call function<br>  add $s0, $v0, $0  # y = returned value<br>  . . .<br>diffofsums:<br>  add $t0, $a0, $a1  # $t0 = f + g<br>  add $t1, $a2, $a3  # $t1 = h + i<br>  sub $s0, $t0, $t1  # result = (f + g) − (h + i)<br>  add $v0, $s0, $0  # put return value in $v0<br>  **jr** $ra  # return to caller |

# 4.6 Function Calls

## The Stack




- The stack is memory that is used to *save local variables within a function*.
- The stack may expand or shrink depending on the processor usage.
- The stack is a last-in-first-out (LIFO) queue.
  The last item **pushed** onto the stack is the first one that can be **popped** off.
- Each function may allocate stack space to store local variables but must deallocate it before returning.
- The top of the stack is the most recently allocated space.
- The stack expands to lower memory addresses when a program needs more space.

# 4.6 Function Calls

## The Stack

Example: Save the contents inside $t0, $t1, and $s0 in the last example.

| MIPS Code (Without Stack) | MIPS Assembly Code (With Stack) |
|---|---|
| diffofsums:<br>add $t0, $a0, $a1<br>add $t1, $a2, $a3<br>sub $s0, $t0, $t1<br>add $v0, $s0, $0<br>**jr** $ra | # $s0 = result<br>diffofsums:<br>addi $sp, $sp, −12 # do space on stack to store 3 regs<br>**sw** $s0, 8($sp) # save $s0 on stack<br>**sw** $t0, 4($sp) # save $t0 on stack<br>**sw** $t1, 0($sp) # save $t1 on stack<br>add $t0, $a0, $a1 # $t0 = f + g<br>add $t1, $a2, $a3 # $t1 = h + i<br>sub $s0, $t0, $t1 # result = (f + g) − (h + i)<br>add $v0, $s0, $0 # put return value in $v0<br>**lw** $t1, 0($sp) # restore $t1 from stack<br>**lw** $t0, 4($sp) # restore $t0 from stack<br>**lw** $s0, 8($sp) # restore $s0 from stack<br>addi $sp, $sp, 12 # deallocate stack space<br>jr $ra # return to caller |

# 4.6 Function Calls

## Preserved Registers

- MIPS divides registers into preserved and nonpreserved categories.
- The preserved registers include $s0–$s7 (saved).
- The nonpreserved registers include $t0–$t9 (temporary).
- A function must save and restore any of the preserved registers that it wishes to use, but it can change the nonpreserved registers freely.

| Preserved | Nonpreserved |
|---|---|
| Saved registers: $s0–$s7 | Temporary registers: $t0–$t9 |
| Return address: $ra | Argument registers: $a0–$a3 |
| Stack pointer: $sp | Return value registers: $v0–$v1 |

# 4.6 Function Calls

Preserved Registers

Example: Save only the contents inside $s0 in the last example.

| MIPS Code (Without Stack) | MIPS Assembly Code (With Stack, Store only preserved reg) |
|---|---|
| diffofsums:<br>add $t0, $a0, $a1<br>add $t1, $a2, $a3<br>sub $s0, $t0, $t1<br>add $v0, $s0, $0<br>**jr** $ra | # $s0 = result<br>diffofsums<br>addi $sp, $sp, −4 # make space on stack to store 1 reg<br>**sw** $s0, 0($sp) # save $s0 on stack<br>add $t0, $a0, $a1 # $t0 = f + g<br>add $t1, $a2, $a3 # $t1 = h + i<br>sub $s0, $t0, $t1 # result = (f + g) − (h + i)<br>add $v0, $s0, $0 # put return value in $v0<br>**lw** $s0, 0($sp) # restore $s0 from stack<br>addi $sp, $sp, 4 # deallocate stack space<br>jr $ra # return to caller |

# 4.6 Function Calls

## Recursive Function Calls

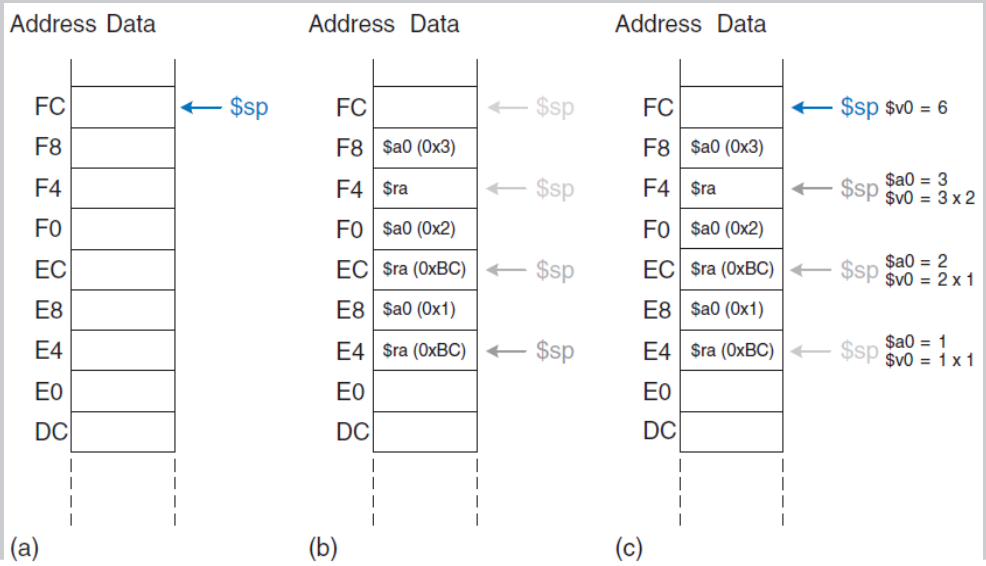
- A function is called *a leaf function* if it doesn't call others, otherwise it's *a nonleaf function*.
- The nonleaf functions are more complicated as they may need to save nonpreserved registers on the stack before calling another function.
- A recursive function is a nonleaf function that calls itself.
- Example: The factorial function is a recursive function.
  Recall that factorial(n) = n × (n – 1) × (n – 2) × . . . × 2 × 1.
  The factorial function can be rewritten recursively as factorial(n) = n × factorial(n – 1).

# 4.6 Function Calls

## Recursive Function Calls

Example: The factorial function .

| High-Level Code |
|---|
| int factorial(int n) {<br>if (n <= 1)<br>  return 1;<br>else<br>  return (n * factorial(n − 1));<br>} |




| MIPS Assembly Code | | |
|---|---|---|
| 0x90 | factorial: addi $sp, $sp, −8 | # do space on stack |
| 0x94 | **sw** $a0, 4($sp) | # store $a0 |
| 0x98 | **sw** $ra, 0($sp) | # store $ra |
| 0x9C | addi $t0, $0, 2 | # $t0 = 2 |
| OxAO | slt $t0, $a0, $t0 | # n <= 1 ? |
| 0xA4 | beq $t0, $0, else | # no: goto else |
| 0xA8 | addi $v0, $0, 1 | # yes: return 1 |
| OxAC | addi $sp, $sp, 8 | # restore $sp |
| OxBO | jr $ra | # return |
| 0xB4 | else: addi $a0, $a0, −1 | #n= n − 1 |
| 0xB8 | jal factorial | # recursive call |
| OxBC | **lw** $ra, 0($sp) | # restore $ra |
| OxCO | **lw** $a0, 4($sp) | # restore $a0 |
| 0xC4 | addi $sp, $sp, 8 | # restore $sp |
| 0xC8 | mul $v0, $a0, $v0 | # n * factorial(n−1) |
| OxCC | jr $ra | # return |

# 6. Memory Map & Program Start

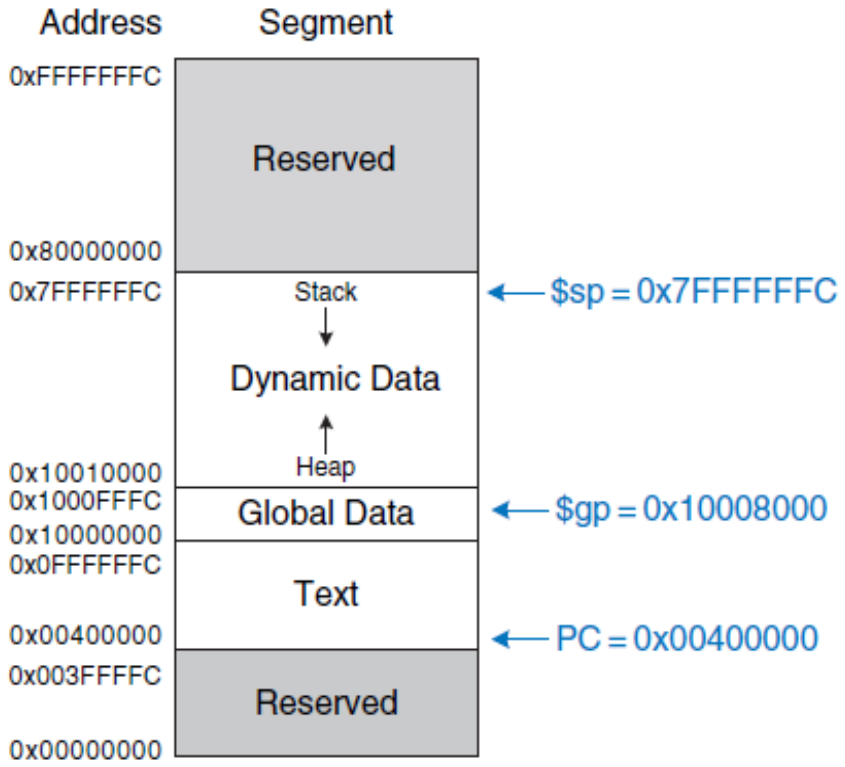
# 6. Memory Map & Program Start


# 6.1 The Memory Map

# 6.2 Translating and Starting a Program

# 6.1 The Memory Map

- With 32-bit addresses, the MIPS address space spans $2^{32}$ bytes = 4 gigabytes (GB).
- Word addresses are divisible by 4 and range from 0 to 0xFFFFFFFC.
- The MIPS architecture divides the address space into 4 parts or segments: *the text segment*, *global data segment*, *dynamic data segment*, and *reserved segments*.

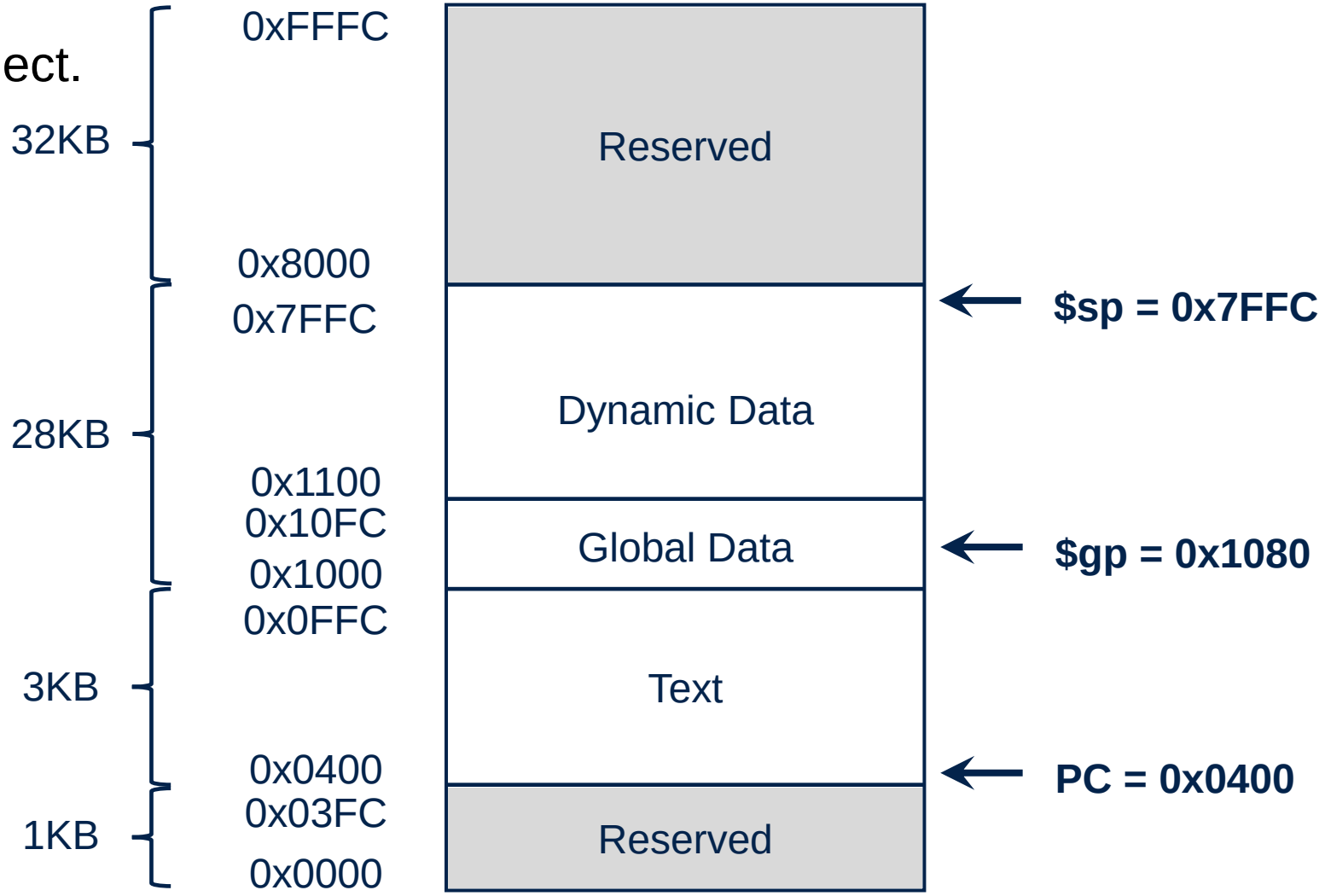



## The Text Segment

- It stores the machine language program.
- It is large to accommodate almost 256 MB of code.
- Note that the four most significant bits of the address in the text space are all 0, so the j instruction can directly jump to any address in the program.

# 6.1 The Memory Map

- Memory Model used for Our Project.




**Exception Handler Address = 0x8180**

# 6.1 The Memory Map

## The Global Data Segment

- The global data segment stores *global variables* which is seen by all functions.
- Global variables are defined at start-up, before the program begins executing.
- The global data segment is large enough to store 64 KB of global variables.
- Global variables are accessed using the global pointer ($gp), initially 0x100080000.
- Unlike the stack pointer ($sp), $gp does not change during program execution.
- Any global variable can be accessed with a 16-bit positive or negative offset from $gp. The offset is known at assembly time, so the variables can be efficiently accessed using base addressing mode with constant offsets.

# 6.1 The Memory Map

## The Dynamic Data Segment

- The dynamic data segment holds the **stack** and the **heap**.
- The data in this segment are not known at start-up but are dynamically allocated and deallocated throughout the execution of the program.
- This is the largest segment of memory used by a program, almost 2 GB.
- The **stack** is used to save and restore registers used by functions and to hold local variables such as arrays.
- The stack grows downward from the top of the dynamic data segment (0x7FFFFFFC) and each stack frame is accessed in last-in-first-out order.

# 6.1 The Memory Map

- The **heap** stores data that is allocated by the program during runtime.
- Like new in C++ and Java which is used to allocate memory.
- The heap grows upward from the bottom of the dynamic data segment.
- If the stack and heap ever grow into each other, the program's data can become corrupted. The *memory allocator* tries to ensure that this never happens.

## The Reserved Segments

- The reserved segments are used by the operating system and cannot directly be used by the program.
- Part of the reserved memory is used for interrupts and for memory-mapped I/O

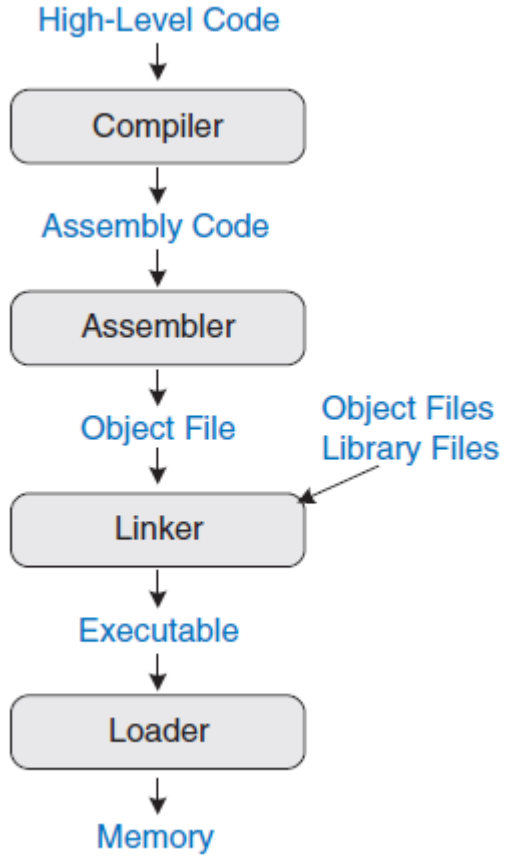
# 6. Memory Map & Program Start


# 6.1 The Memory Map

# 6.2 Translating and Starting a Program

# 6.2 Translating and Starting a Program

- Four steps are required to translate a program from a high-level language into machine language and to start executing that program.
- First, the high-level code is compiled into assembly code.
- The assembly code is assembled into machine code in an object file.
- The linker combines the machine code with object code from libraries and other files to produce an entire executable program.
- In practice, most compilers perform all three steps of compiling, assembling, and linking.
- Finally, the loader loads the program into memory and starts execution.



# 6.2 Translating and Starting a Program

## Step 1: Compilation

- A compiler translates high-level code into assembly language.

Example: a simple high-level program with three global variables and two functions, along with the assembly code produced by a typical compiler.

| High-Level Code | MIPS Assembly Code |
|---|---|
| int f, g, y; // global variables<br>int main(void)<br>{<br>f = 2;<br>g = 3;<br>y = sum(f, g);<br>return y;<br>}<br>int sum(int a, int b) {<br>return (a + b);<br>} | .data<br>f:<br>g:<br>y:<br>.text<br>main: addi $sp, $sp, −4 # make stack frame<br>sw $ra, 0($sp) # store $ra on stack<br>addi $a0, $0, 2 # $a0 = 2<br>sw $a0, f # f = 2<br>..<br>jal sum # call sum function<br>..<br>jr $ra # return to operating system<br>sum: add $v0, $a0, $a1 # $v0 = a + b<br>jr $ra # return to caller |

# 6.2 Translating and Starting a Program

## Step 2: Assembling

- The assembler turns the assembly code into an object file containing machine code.
- The assembler assigns instruction addresses and finds all the symbols, such as labels and global variable names.
- The names and addresses of the symbols are kept in a symbol table.
- Global variables are assigned storage locations in the global data segment of memory, starting at memory address 0x10000000.

# 6.2 Translating and Starting a Program

```
0x00400000  main: addi $sp, $sp, −4
0x00400004  sw $ra, 0($sp)
0x00400008  addi $a0, $0, 2
0x0040000C  sw $a0, f
0x00400010  addi $a1, $0, 3
0x00400014  sw $a1, g
0x00400018  jal sum
0x0040001C  sw $v0, y
0x00400020  lw $ra, 0($sp)
0x00400024  addi $sp, $sp, 4
0x00400028  jr $ra
0x0040002C  sum: add $v0, $a0, $a1
0x00400030  jr $ra
```

| Symbol | Address |
|---|---|
| f | 0x10000000 |
| g | 0x10000004 |
| y | 0x10000008 |
| main | 0x00400000 |
| sum | 0x0040002C |

- Then, the assembler produces the machine language code.
  Addresses for the global variables and labels are taken from the symbol table.
  The machine language code and symbol table are stored in the object file.

# 6.2 Translating and Starting a Program

## Step 3: Linking

- Most large programs contain more than one file. If the programmer changes only one of the files, the associated object file only will be updated.
- The linker combines all the object files into one machine language file (the executable).
- The executable file has three sections: the executable file header, the text segment, and the data segment.
- The executable file header reports the text size (code size) and data size (global data).
- The text segment gives the instructions which are stored in memory.
- The data segment addresses all global variables.

| Executable file header | Text Size | Data Size |
|---|---|---|
| | 0x34 (52 bytes) | 0xC (12 bytes) |
| **Text segment** | **Address** | **Instruction** |
| | 0x00400000 | 0x23BDFFFC |
| | 0x00400004 | 0xAFBF0000 |
| | 0x00400008 | 0x20040002 |
| | 0x0040000C | 0xAF848000 |
| | 0x00400010 | 0x20050003 |
| | 0x00400014 | 0xAF858004 |
| | 0x00400018 | 0x0C10000B |
| | 0x0040001C | 0xAF828008 |
| | 0x00400020 | 0x8FBF0000 |
| | 0x00400024 | 0x23BD0004 |
| | 0x00400028 | 0x03E00008 |
| | 0x0040002C | 0x00851020 |
| | 0x00400030 | 0x03E00008 |
| **Data segment** | **Address** | **Data** |
| | 0x10000000 | f |
| | 0x10000004 | g |
| | 0x10000008 | y |

```
addi $sp, $sp, –4
sw   $ra, 0($sp)
addi $a0, $0, 2
sw   $a0, 0x8000($gp)
addi $a1, $0, 3
sw   $a1, 0x8004($gp)
jal  0x0040002C
sw   $v0, 0x8008($gp)
lw   $ra, 0($sp)
addi $sp, $sp, –4
jr   $ra
add  $v0, $a0, $a1
jr   $ra
```

# 6.2 Translating and Starting a Program

Step 3: Linking

- The global variables are addressed with respect to the base address given by the global pointer, $gp.

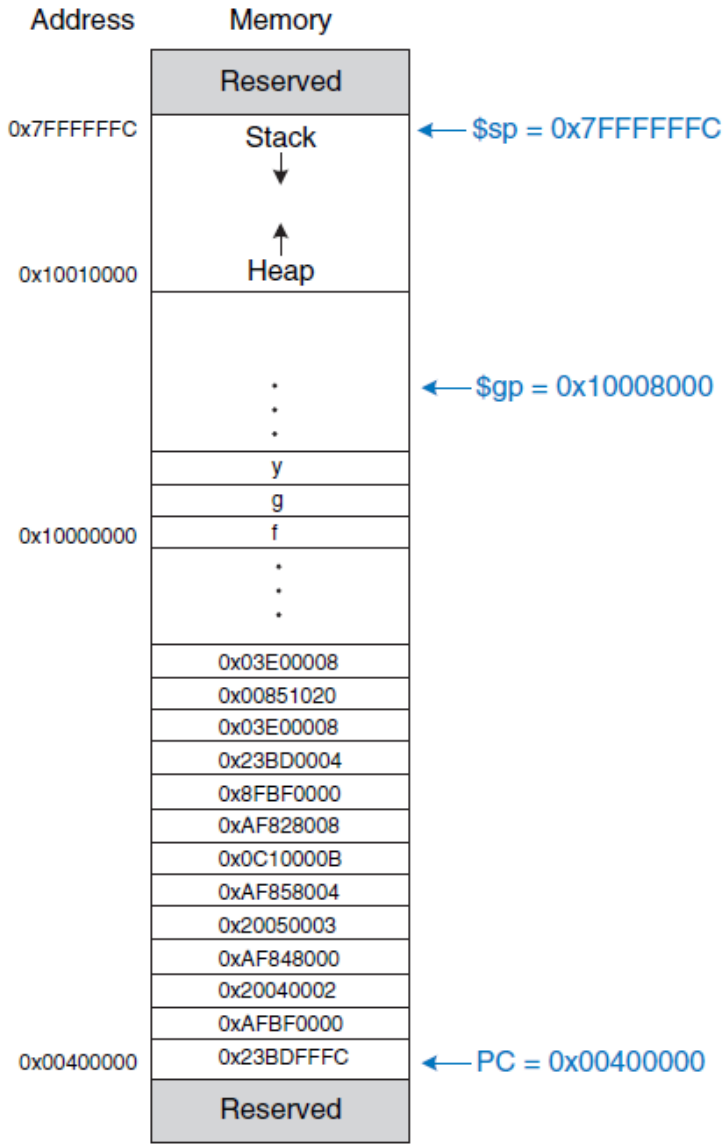
Example: the first store instruction, sw $a0, 0x8000($gp), stores 2 to the global variable f.

- The base address, $gp = 0x10008000.
- The offset, 0x8000, is a 16-bit signed number that is sign-extended = 0xFFFF8000.
- So, the memory address of variable f = 0x10008000 + 0xFFFF8000 = 0x10000000.

# 6.2 Translating and Starting a Program

## Step 4: Loading

- The operating system loads a program by reading the text segment of the executable file from a storage device (usually the hard disk) into the text segment of memory.
- The operating system sets $gp to 0x10008000 (the middle of the global data segment) and $sp to 0x7FFFFFFC (the top of the dynamic data segment).
- Then, OS performs a jal 0x00400000 to jump to the beginning of the program.



# Assignment #2

# Assignment #2

- Read section *7.3 SingleCycle Processor* from the textbook.
- Implement the **full** SingleCycle MIPS Processor which supports all the mentioned instructions, documented in *Appendix B, except floating point instructions*.
- Review and update the single-Cycle MIPS processor.
- Apply the given tests [Test#1,2,5] to verify the functionality.
- [As a plus] Apply a randomized testbench, do the required constraints and the possible assertions
- Deadline: Friday, 1st Nov 2024, at 11:59 PM.
- Deliveries: Design files, testbench, and a presentation for your work.
  Send a mail to: eng.abdulkareem.abotaleb@gmail,com
  Using a title of: CompArch_assignment2_<first_name>_<last_name>

# Thank You